

TriviUp - Manual Técnico

Documentación técnica para desarrolladores y administradores del sistema

TriviUp - Manual Técnico	1
1. Visión General	3
1.1 Propósito	3
1.2 Stack Tecnológico	3
1.3 URLs de Producción	3
2. Arquitectura del Sistema	4
2.1 Diagrama de Arquitectura	4
2.2 Flujo de Datos	4
2.3 Estructura de Archivos	5
3. Backend - .NET 9	5
3.1 Punto de Entrada (Program.cs)	5
3.2 Inyección de Dependencias	6
3.3 Controladores	7
AuthController	7
UsersController	8
CuestionariosController	9
3.4 SignalR Hub (GameHub)	10
3.5 Manejo de Errores	11
3.6 CORS Configuration	12
4. Frontend - Angular 21	13
4.1 Configuración de la Aplicación	13
4.2 Routing	13
4.3 Servicios Principales	15
AuthService (Signal-based)	15
GameSignalrService (SignalR + Signals)	16
4.4 Interceptores	17
4.5 Guards	18
4.6 Componentes de Juego	19
GameRoomComponent	19
5. Base de Datos	21
5.1 Diagrama ER	21
5.2 EF Core Entities	22
5.4 DbContext	23
6. Sistema de Autenticación	25

6.1 Flujo de Autenticación	25
6.2 JWT Token Structure	25
6.3 Google OAuth Flow	26
6.4 Autorización en SignalR	26
7. Comunicación en Tiempo Real	28
7.1 SignalR Configuration	28
7.2 GameRoom (Estado en Memoria)	28
7.3 Flujo de Partida	30
7.4 Reconexión	30
8. Servicios y Configuración	31
8.1 Configuración de Services	31
8.2 Storage Service	33
8.3 Cache Service	34
9. Testing	35
9.1 Backend - xUnit + Moq	35
9.2 E2E - Playwright	37
9.3 Ejecutar Tests	38
10. Despliegue	39
10.1 Railway	39
10.2 nginx Configuration	39
10.3 Logs	43
Anexo A: Glosario	43
Anexo B: Referencia de API Endpoints	44

1. Visión General

1.1 Propósito

Este manual técnico proporciona información detallada para:

- **Desarrolladores** que necesiten entender o modificar el código
- **Administradores** que gestionen el despliegue en Railway
- **Nuevos miembros del equipo** que necesiten ponerse al día

1.2 Stack Tecnológico

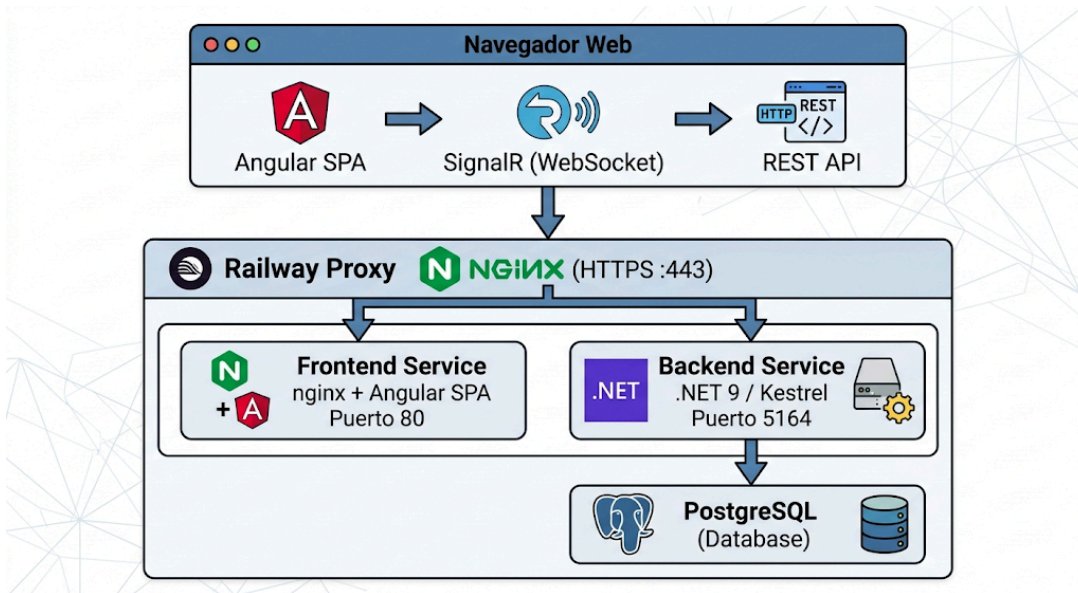
Capa	Tecnología	Versión
Frontend	Angular	21.x
Estilos	TailwindCSS	v4.x
UI Components	DaisyUI	5.x
Backend	.NET	9.x
ORM	Entity Framework Core	9.x
Base de Datos	PostgreSQL	15+
Tiempo Real	SignalR	(incluido en .NET)
Cache	IMemoryCache	(incluido en .NET)
Deployment	Railway	-

1.3 URLs de Producción

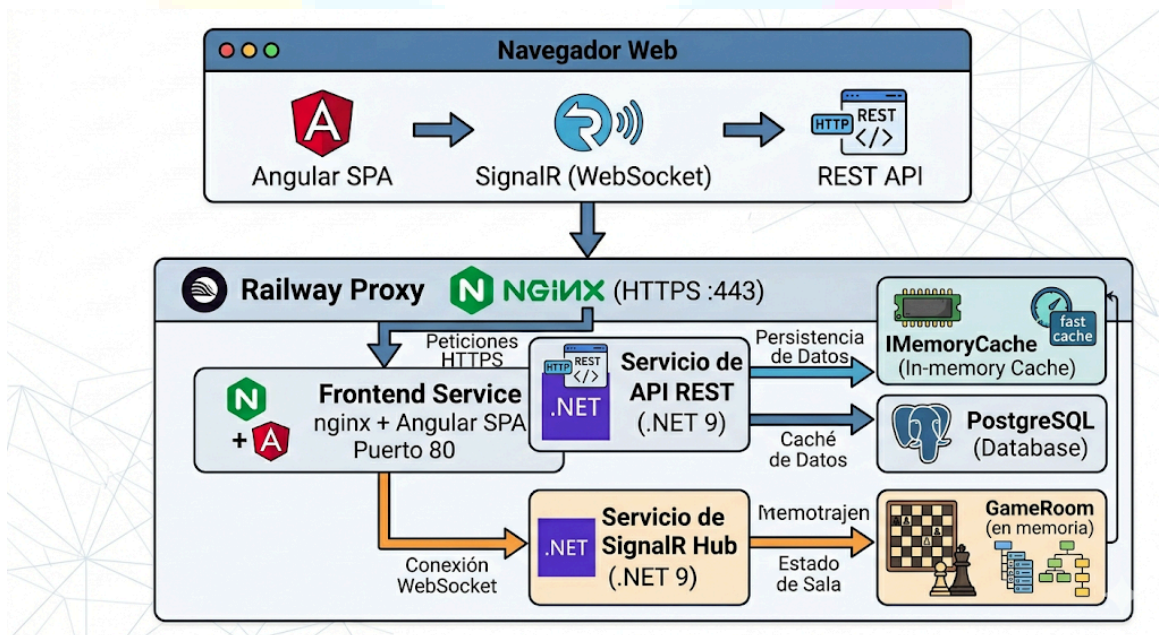
Servicio	URL
Frontend	https://triviup.up.railway.app
Backend API	https://triviup-backend.up.railway.app
Swagger	https://triviup-backend.up.railway.app/swagger

2. Arquitectura del Sistema

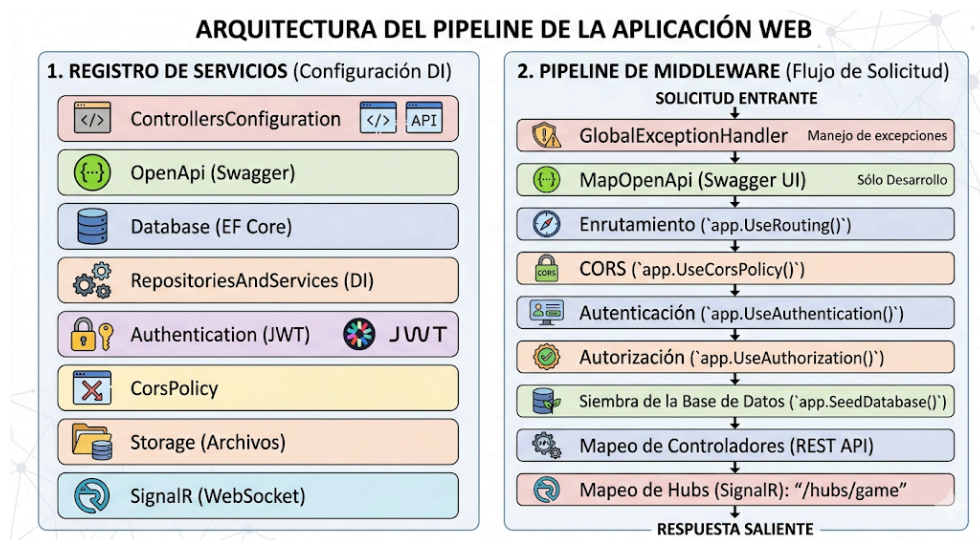
2.1 Diagrama de Arquitectura



2.2 Flujo de Datos



2.3 Estructura de Archivos



3. Backend - .NET 9

3.1 Punto de Entrada (Program.cs)

```
var builder = WebApplication.CreateBuilder(args);

// 1. Registro de servicios

builder.Services.AddControllersConfiguration(); // Controladores

builder.Services.AddOpenApi(); // Swagger

builder.Services.AddDatabase(builder.Configuration); // EF Core

builder.Services.AddRepositoriesAndServices(); // DI

builder.Services.AddAuthentication(); // JWT

builder.Services.AddCorsPolicy(); // CORS
```

```
builder.Services.AddStorage(); // Archivos

builder.Services.AddSignalR(); // WebSocket

// 2. Pipeline de middleware

var app = builder.Build();

app.UseGlobalExceptionHandler(); // Manejo de excepciones

if (app.Environment.IsDevelopment())

    app.MapOpenApi(); // Swagger UI

app.UseRouting();

app.UseCorsPolicy();

app.UseAuthentication();

app.UseAuthorization();

app.SeedDatabase(); // Datos iniciales

app.MapControllers(); // REST API

app.MapHub<GameHub>("/hubs/game"); // SignalR

app.Run();
```

3.2 Inyección de Dependencias

Los servicios se registran en `Infrastructure/ServiceExtensions.cs`:

```
// Repositorios

services.AddScoped<IUserRepository, UserRepository>();

services.AddScoped<IQuizRepository, QuizRepository>();

services.AddScoped<IGameHistoryRepository, GameHistoryRepository>();
```

```
// Servicios

services.AddScoped<IAuthService, AuthService>();

services.AddScoped<IJwtService, JwtService>();

services.AddScoped<IQuizService, QuizService>();

// SignalR y Game

services.AddSingleton<ITurnManager, TurnManager>();

services.AddSingleton<IGameService, GameService>();

services.AddSingleton<GameOptions>();

// Cache

services.AddMemoryCache();

services.AddScoped<ICacheService, CacheService>();
```

3.3 Controladores

AuthController

```
[ApiController]

[Route("auth")]

public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;

    [HttpPost("signup")]

    public async Task<IActionResult> SignUp([FromBody] RegisterDto dto);

    [HttpPost("signin")]
```

```
public async Task<IActionResult> SignIn([FromBody] LoginDto dto);

[HttpGet("google")]

public IActionResult GoogleLogin();

[HttpGet("google/callback")]

public async Task<IActionResult> GoogleCallback(string code, string
state);
}
```

UserController

```
[ApiController]

[Route("users")]

public class UsersController : ControllerBase
{
    // Público (JWT)

    [HttpGet("me")]

    [Authorize]

    public async Task<IActionResult> GetCurrentUser();

    [HttpPut("me")]

    [Authorize]
```

```
public async Task<IActionResult> UpdateCurrentUser([FromBody] UpdateUserDto
dto);
```

```
// Admin

[HttpGet]
```

```
[Authorize(Roles = "ADMIN")]

public async Task<IActionResult> GetAllUsers();

[HttpPut("{id}")]

[Authorize(Roles = "ADMIN")]

public async Task<IActionResult> UpdateUser(long id, [FromBody]
UpdateUserDto dto);

[HttpPost("{id}/ban")]

[Authorize(Roles = "ADMIN")]

public async Task<IActionResult> BanUser(long id);

[HttpPost("{id}/activate")]

[Authorize(Roles = "ADMIN")]

public async Task<IActionResult> ActivateUser(long id);

}
```

CuestionariosController

```
[ApiController]

[Route("api/cuestionarios")]

[Authorize]

public class CuestionariosController : ControllerBase

{

    [HttpPost]

    public async Task<IActionResult> Create([FromBody] CreateQuizRequest
request);

}
```

```
[HttpGet]

    public async Task<IActionResult> GetAll([FromQuery] int page,
[FromQuery] int pageSize);

[HttpGet("{id}")]

    public async Task<IActionResult> GetById(long id);

[HttpGet("mis-cuestionarios")]

    public async Task<IActionResult> GetMine();

[HttpPut("{id}")]

    public async Task<IActionResult> Update(long id, [FromBody]
UpdateQuizRequest request);

[HttpDelete("{id}")]

    public async Task<IActionResult> Delete(long id);

}
```

3.4 SignalR Hub (GameHub)

```
[Authorize]

public class GameHub : Hub

{

    // Cliente —► Servidor

    public async Task CreateGame(long quizId);

    public async Task JoinGame(string roomCode, long? userId, string?
username);

    public async Task LeaveGame(string roomCode);

}
```

```
public async Task StartGame(string roomCode);

public async Task SubmitAnswer(string roomCode, long questionId, int
answerIndex, int timeRemaining);

public async Task PauseGame(string roomCode);

public async Task ResumeGame(string roomCode);

public async Task KickPlayer(string roomCode, long playerId);

// Servidor —► Cliente (invocado desde GameService)

public async Task SendGameCreated(GameLobbyState state);

public async Task SendPlayersList(Player[] players);

public async Task SendPlayerJoined(Player player);

public async Task SendGameStarted(GameStateDto state);

public async Task SendTurnStarted(TurnStartedDto dto);

public async Task SendTurnResult(TurnResult result);

public async Task SendGameFinished(GameResult result);

}
```

3.5 Manejo de Errores

```
// Middleware de excepciones

app.UseGlobalExceptionHandler();

// Errores tipados

public class QuizNotFoundError : QuizError { }

public class QuizForbiddenError : QuizError { }

public class AuthUnauthorizedError : AuthError { }
```

```
// Uso en servicios

return result.Match(

    Ok,

    error => error switch

    {

        QuizNotFoundError e => NotFound(e.Message),

        QuizForbiddenError e => Forbid(e.Message),

        _ => StatusCode(500, "Error inesperado")

    });
```

3.6 CORS Configuration

```
// Infrastructure/CorsExtensions.cs

services.AddCors(options =>

{

    options.AddPolicy("AllowFrontend", policy =>

    {

        policy.WithOrigins(frontendUrl)

            .AllowAnyHeader()

            .AllowAnyMethod()

            .AllowCredentials(); // Requerido para SignalR

    });

});
```

4. Frontend - Angular 21

4.1 Configuración de la Aplicación

```
// app.config.ts

export const appConfig: ApplicationConfig = {

  providers: [

    provideRouter(routes),

    provideHttpClient(withInterceptors([authInterceptor])),

    provideAnimations(),

    provideCharts(config => {

      return new ChartConfiguration(config);

    })

  ]

};
```

4.2 Routing

```
// app.routes.ts

export const routes: Routes = [

  { path: '', component: HomeComponent },

  { path: 'auth', loadComponent: () => AuthComponent },

  { path: 'auth/callback', loadComponent: () => AuthCallbackComponent },

  {

    path: 'cuestionarios',
```

```
    canActivate: [authGuard],

    children: [

      { path: 'crear', loadComponent: () => CrearCuestionarioPage },

      { path: 'mis-cuestionarios', loadComponent: () =>
QuizListComponent },

      { path: ':id', loadComponent: () => QuizDetailComponent }

    ]

  },

  { path: 'quizzes/public', loadComponent: () => QuizzesPublicComponent
},

  { path: 'game/:roomCode', loadComponent: () => GameRoomComponent },

  { path: 'unirse', loadComponent: () => JoinRoomComponent },

  { path: 'admin', canActivate: [isAdminGuard], loadComponent: () =>
AdminDashboardComponent },

  { path: 'admin/users', canActivate: [isAdminGuard], loadComponent: ()
=> AdminUsersComponent },

  { path: 'perfil', canActivate: [authGuard], loadComponent: () =>
ProfileComponent },

  { path: 'perfil/editar', canActivate: [authGuard], loadComponent: () =>
ProfileEditComponent },

  { path: 'game/history', canActivate: [authGuard], loadComponent: () =>
GameHistoryComponent },

  { path: 'game/history/:gameId', canActivate: [authGuard],
loadComponent: () => GameDetailsComponent },

  { path: 'informacion', loadComponent: () => InformacionComponent },
```

```
{ path: 'contacto', loadComponent: () => ContactoComponent },  
  
{ path: '**', redirectTo: '' }  
  
];
```

4.3 Servicios Principales

AuthService (Signal-based)

```
@Injectable({ providedIn: 'root' })  
  
export class AuthService {  
  
    private readonly USER_KEY = 'triviup_user';  
  
    private readonly TOKEN_KEY = 'triviup_token';  
  
    // Signals  
  
    private _userSignal = signal<AuthUser | null>(null);  
  
    readonly userSignal = this._userSignal.asReadonly();  
  
    // Computed  
  
    readonly isLoggedIn = computed(() => this._userSignal() !== null);  
  
    readonly isAdmin = computed(() => this._userSignal()?.role ===  
    'ADMIN');  
  
    // Methods  
  
    signIn(payload: SignInPayload): Observable<AuthResponse>;  
  
    signUp(payload: SignUpPayload): Observable<AuthResponse>;  
  
    saveSession(response: AuthResponse): void;  
  
    logout(): void;
```

```
getToken(): string | null;  
  
getUser(): AuthUser | null;  
  
}
```

GameSignalrService (SignalR + Signals)

```
@Injectable({ providedIn: 'root' })  
  
export class GameSignalrService {  
  
    // Connection  
  
    private connection: signalR.HubConnection | null = null;  
  
    // Signals  
  
    readonly isConnected = signal(false);  
  
    readonly currentRoomCode = signal<string | null>(null);  
  
    readonly players = signal<Player[]>([]);  
  
    readonly currentQuestion = signal<Question | null>(null);  
  
    readonly currentTurnPlayerId = signal<number | null>(null);  
  
    readonly timeRemaining = signal(21);  
  
    readonly isMyTurn = signal(false);  
  
    readonly gameResults = signal<GameResult | null>(null);  
  
    readonly isPaused = signal(false);  
  
    readonly isOwner = signal(false);  
  
    // Event emitters (observables para eventos)  
  
    readonly onPlayerJoined = new Subject<Player>();  
  
}
```

```
readonly onTurnStarted = new Subject<TurnStartedDto>();

readonly onTurnResult = new Subject<TurnResult>();

readonly onGameFinished = new Subject<GameResult>();

// Methods

async connect(token: string): Promise<void>;

async disconnect(): Promise<void>;

async createGame(quizId: number): Promise<string>;

async joinGame(roomCode: string, userId?: number, username?: string):
Promise<void>;

async startGame(roomCode: string): Promise<void>;

async submitAnswer(roomCode: string, questionId: number, answerIndex:
number, timeRemaining: number): Promise<void>;

async pauseGame(roomCode: string): Promise<void>;

async resumeGame(roomCode: string): Promise<void>;

async kickPlayer(roomCode: string, playerId: number): Promise<void>;

}
```

4.4 Interceptores

```
// auth.interceptor.ts

export const authInterceptor: HttpInterceptorFn = (req, next) => {

  const authService = inject(AuthService);

  const token = authService.getToken();

  if (token && req.url.includes('/api/')) {
```

```
    const authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });

    return next(authReq);
  }

  return next(req);
};
```

4.5 Guards

```
// auth.guard.ts

export const authGuard: CanActivateFn = () => {

  const authService = inject(AuthService);

  const router = inject(Router);

  if (authService.isLoggedIn()) {

    return true;

  }

  router.navigate(['/auth']);

  return false;

};

// is-admin.guard.ts

export const isAdminGuard: CanActivateFn = () => {

  const authService = inject(AuthService);
```

```
const router = inject(Router);

if (authService.isAdmin()) {

    return true;

}

router.navigate(['/']);

return false;

};
```

4.6 Componentes de Juego

GameRoomComponent

```
@Component({

  selector: 'app-game-room',

  standalone: true,

  template: `

    @if (isInGame()) {

      <div class="game-container">

        <!-- Players sidebar -->

        <app-players-list
[players]="gameService.players()"></app-players-list>

        <!-- Main game area -->

        <div class="game-area">

          @if (gameService.isMyTurn()) {

            <app-question-card
```

```
        [question]="gameService.currentQuestion()"
        (answerSelected)="onAnswerSelected($event)">
    </app-question-card>

    } @else {

        <app-waiting-turn

            [currentPlayer]="getCurrentTurnPlayer()"

            [timeRemaining]="gameService.timeRemaining()">

        </app-waiting-turn>

    }

</div>

<!-- Owner controls -->

@if (gameService.isOwner()) {

    <app-owner-controls

        [isPaused]="gameService.isPaused()"

        (pause)="pauseGame()"

        (resume)="resumeGame()"

        (kick)="kickPlayer($event)">

    </app-owner-controls>

}

</div>

}
```

```
})

export class GameRoomComponent {

  isInGame = computed(() => this.gameService.currentRoomCode() !== null);

  constructor(private gameService: GameSignalrService) {

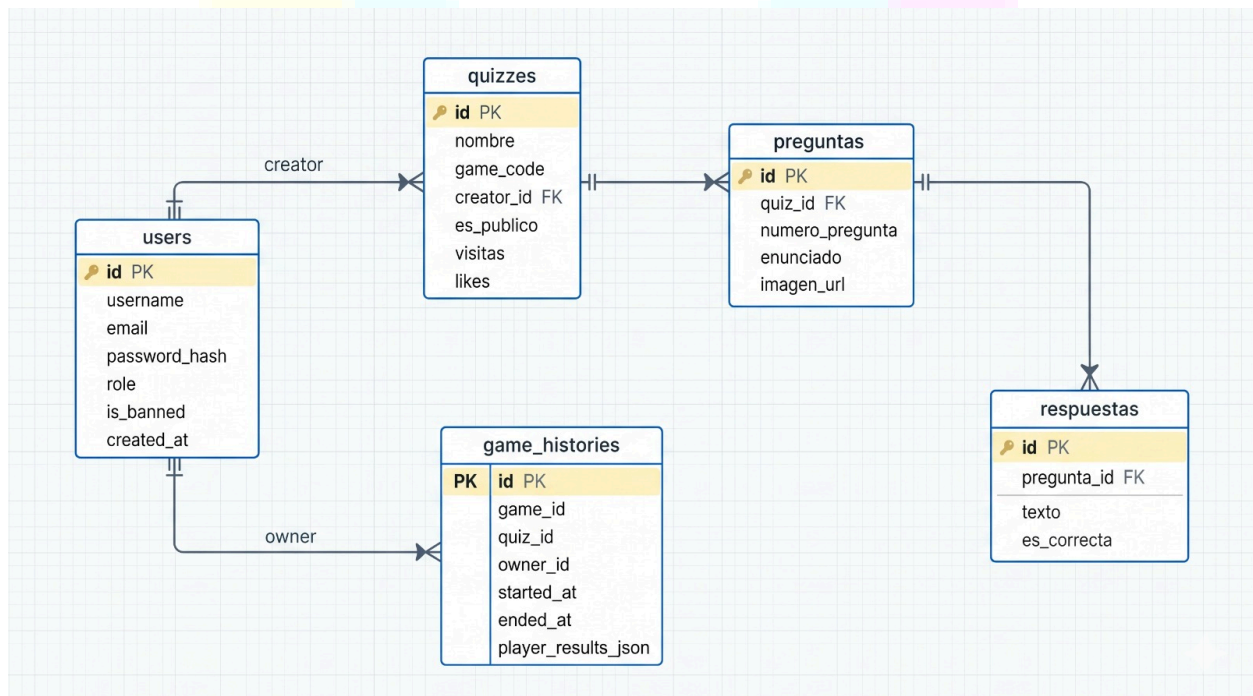
    // Subscribe to SignalR events

  }

}
```

5. Base de Datos

5.1 Diagrama ER



5.2 EF Core Entities

```
// Entities/User.cs

public class User
{
    public long Id { get; set; }

    public string Username { get; set; }

    public string Email { get; set; }

    public string? PasswordHash { get; set; }

    public string Role { get; set; } = "USER";

    public bool IsDeleted { get; set; }

    public bool IsBanned { get; set; }

    public DateTime CreatedAt { get; set; }

    public DateTime UpdatedAt { get; set; }

    public string? GoogleId { get; set; }

    public string? ProfilePhotoUrl { get; set; }

    public DateTime? LastLoginAt { get; set; }

    // Navigation

    public ICollection<Quiz> Quizzes { get; set; }
}

// Entities/Quiz.cs

public class Quiz
```

```
{  
  
    public long Id { get; set; }  
  
    public string Nombre { get; set; }  
  
    public string GameCode { get; set; }  
  
    public long CreatorId { get; set; }  
  
    public bool EsPublico { get; set; }  
  
    public int Visitas { get; set; }  
  
    public int Likes { get; set; }  
  
    public DateTime CreatedAt { get; set; }  
  
    public DateTime UpdatedAt { get; set; }  
  
    // Navigation  
  
    public User Creator { get; set; }  
  
    public ICollection<Pregunta> Preguntas { get; set; }  
  
}
```

5.4 DbContext

```
// Database/AppDbContext.cs  
  
public class AppDbContext : DbContext  
  
{  
  
    public DbSet<User> Users { get; set; }  
  
    public DbSet<Quiz> Quizzes { get; set; }  
  
    public DbSet<Pregunta> Preguntas { get; set; }  
  
}
```

```
public DbSet<Respuesta> Respuestas { get; set; }

public DbSet<GameHistory> GameHistories { get; set; }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Soft delete filter

    modelBuilder.Entity<User>()

        .HasQueryFilter(u => !u.IsDeleted);

    // Unique constraints

    modelBuilder.Entity<User>()

        .HasIndex(u => u.Email)

        .IsUnique();

    modelBuilder.Entity<Quiz>()

        .HasIndex(q => q.GameCode)

        .IsUnique();

    // Cascade delete

    modelBuilder.Entity<Pregunta>()

        .HasOne(p => p.Quiz)

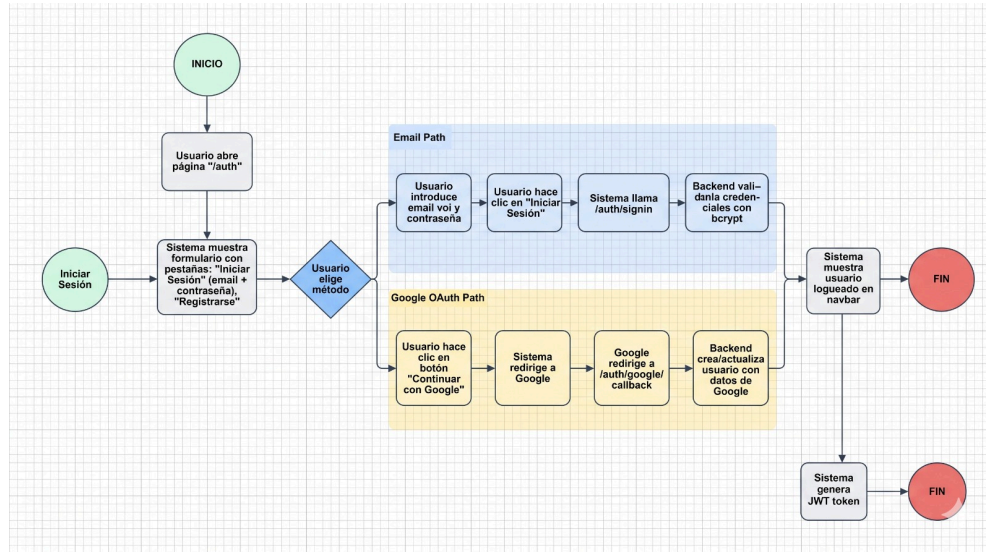
        .WithMany(q => q.Preguntas)

        .HasForeignKey(p => p.QuizId)

        .OnDelete(DeleteBehavior.Cascade);
}
```

6. Sistema de Autenticación

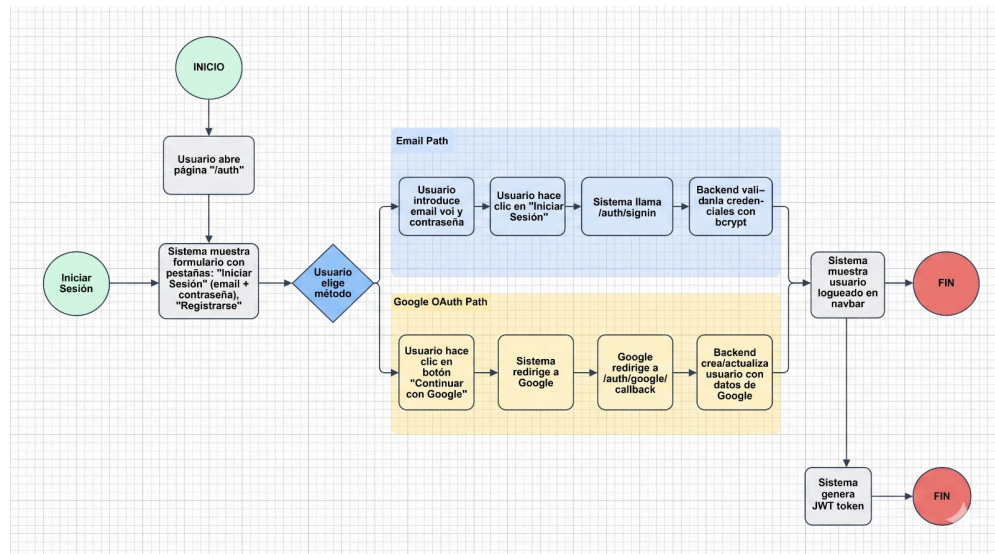
6.1 Flujo de Autenticación



6.2 JWT Token Structure

```
{
  "sub": "123",
  "email": "user@example.com",
  "role": "USER",
  "username": "jugador1",
  "jti": "uuid-unico",
  "iss": "TriviUpBackend",
  "aud": "TriviUpApp",
  "exp": 1714567890,
  "iat": 1714564290
}
```

6.3 Google OAuth Flow



6.4 Autorización en SignalR

```
// GameHub.cs

public override async Task OnConnectedAsync()
{
    var token = Context.GetHttpContext()
        .Request.Query["token"].FirstOrDefault();

    if (string.IsNullOrEmpty(token))
    {
        // Anonymous connection allowed for joining games

        return;
    }

    var userId = _jwtTokenExtractor.ExtractUserId(token);

    if (userId.HasValue)
```

```
{  
    // Authenticated user  
    Context.Items["UserId"] = userId.Value;  
}  
}  
  
// En métodos que requieren auth  
public async Task CreateGame(long quizId)  
{  
    var userId = Context.Items["UserId"] as long?;  
    if (!userId.HasValue)  
        throw new HubException("Unauthorized");  
  
    // Validar que el usuario existe y no está banned  
    var user = await _userRepository.FindByIdAsync(userId.Value);  
    if (user == null || user.IsBanned)  
        throw new HubException("User not allowed");  
}
```

7. Comunicación en Tiempo Real

7.1 SignalR Configuration

```
// Program.cs

builder.Services.AddSignalR(options =>
{
    options.EnableDetailedErrors = true;

    options.MaximumReceiveMessageSize = 1024 * 1024; // 1MB
});

// Configuración de reconexión del cliente

const connection = new signalR.HubConnectionBuilder()

    .withUrl('/hubs/game', { accessTokenFactory: () => token })

    .withAutomaticReconnect({

        0: 10000,    // 10 segundos

        1: 30000,    // 30 segundos

        2: 60000,    // 1 minuto

    })

    .build();
```

7.2 GameRoom (Estado en Memoria)

```
// Game/Models/GameRoom.cs

public class GameRoom

{
```

```
public string RoomCode { get; set; }

public long QuizId { get; set; }

public string QuizTitle { get; set; }

public long OwnerId { get; set; }

public GameState State { get; set; }

public List<Player> Players { get; set; } = new();

public Queue<long> TurnOrder { get; set; } = new();

public int CurrentQuestionIndex { get; set; }

public List<Pregunta> Questions { get; set; } = new();

public DateTime CreatedAt { get; set; }

public DateTime? StartedAt { get; set; }

public DateTime? EndedAt { get; set; }

public DateTime? TurnStartedAt { get; set; }

public int? PausedTimeRemaining { get; set; }

}

public enum GameState

{

    Waiting,

    Starting,

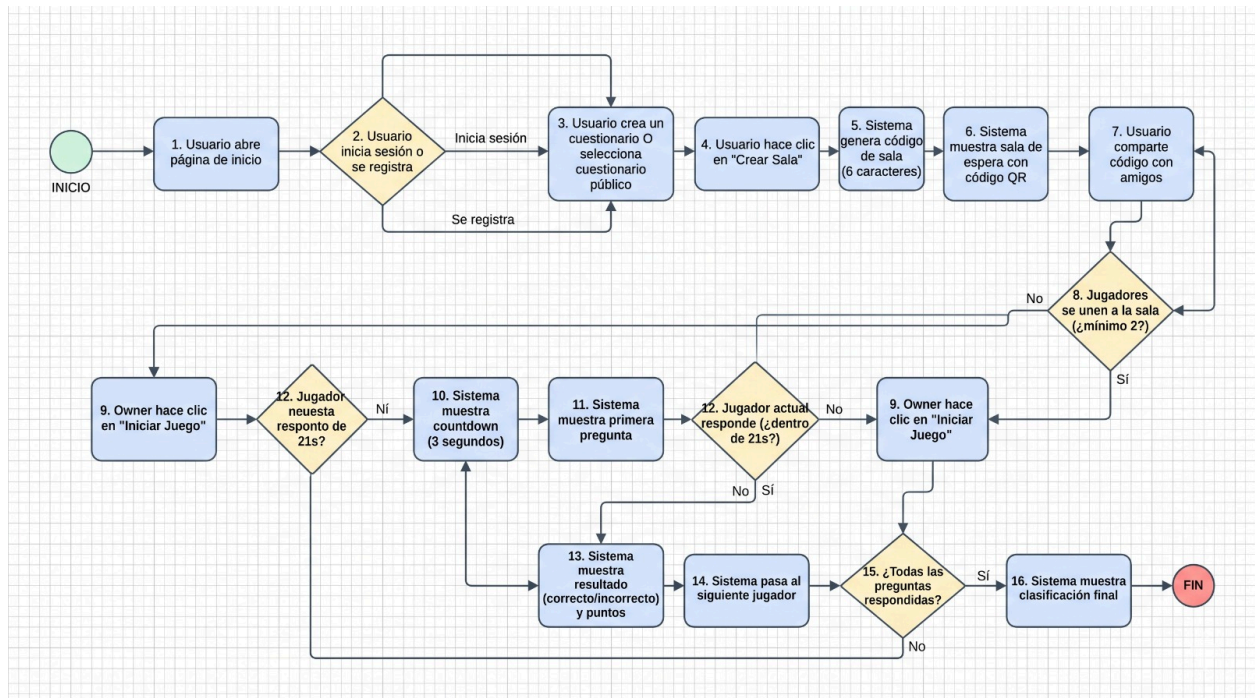
    Playing,

    Paused,
```

Finished

}

7.3 Flujo de Partida



7.4 Reconexión

```
// Cliente: evento de reconexión

connection.onreconnecting((error) => {

    console.log("Conexión perdida. Reconectando...", error);

    this.isConnected.set(false);

});

connection.onreconnected((connectionId) => {

    console.log("Reconectado:", connectionId);

    this.isConnected.set(true);

});
```

```
// Re-join room si estaba en una

if (this.currentRoomCode()) {

    this.joinGame(this.currentRoomCode());

}

});
```

8. Servicios y Configuración

8.1 Configuración de Services

```
// Infrastructure/ServiceExtensions.cs

public static class ServiceExtensions
{

    public static IServiceCollection AddDatabase(

        this IServiceCollection services,

        IConfiguration configuration)

    {

        var provider = configuration["Database:Provider"];

        if (provider == "PostgreSQL")

        {

            services.AddDbContext<AppDbContext>(options =>

                options.UseNpgsql(connectionString));

        }

    }

}
```

```
    }

    else

    {

        // InMemory para testing

        services.AddDbContext<AppDbContext>(options =>

            options.UseInMemoryDatabase("TriviUpDb"));

    }

    return services;
}

public static IServiceCollection AddJwtAuthentication(

    this IServiceCollection services,

    IConfiguration configuration)

{

    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)

        .AddJwtBearer(options =>

            {

                options.TokenValidationParameters = new

TokenValidationParameters

                {

                    ValidateIssuer = true,

                    ValidateAudience = true,

                    ValidateLifetime = true,
```

```
        ValidateIssuerSigningKey = true,  
        ValidIssuer = configuration["Jwt:Issuer"],  
        ValidAudience = configuration["Jwt:Audience"],  
        IssuerSigningKey = new SymmetricSecurityKey(  
            Encoding.UTF8.GetBytes(configuration["Jwt:Key"]))  
    };  
});  
  
return services;  
}  
}
```

8.2 Storage Service

```
// Common/Storage/StorageService.cs  
  
public interface IStorageService  
{  
    string GetFullPath(string relativePath);  
    bool IsValidPath(string path);  
}  
  
public class LocalStorageService : IStorageService  
{  
    private readonly string _uploadPath;  
  
    public LocalStorageService(IConfiguration configuration)
```

```
{  
    _uploadPath = configuration["Storage:UploadPath"] ?? "uploads";  
}  
  
public string GetFullPath(string relativePath)  
    => Path.Combine(_uploadPath, relativePath);  
  
public bool IsValidPath(string path)  
{  
    var fullPath = GetFullPath(path);  
  
    return fullPath.Contains(_uploadPath); // Prevent directory  
traversal  
}  
}}
```

8.3 Cache Service

```
// Services/Cache/CacheService.cs  
  
public interface ICacheService  
{  
    Task<T?> GetAsync<T>(string key);  
  
    Task SetAsync<T>(string key, T value, TimeSpan? expiry = null);  
  
    Task RemoveAsync(string key);  
  
    Task RemoveByPrefixAsync(string prefix);  
}  
  
public class MemoryCacheService : ICacheService
```

```
{  
  
    private readonly IMemoryCache _cache;  
  
    public async Task<T?> GetAsync<T>(string key)  
    {  
        _cache.TryGetValue(key, out T? value);  
  
        return value;  
    }  
  
    public async Task SetAsync<T>(string key, T value, TimeSpan? expiry =  
null)  
    {  
        var options = new MemoryCacheEntryOptions();  
  
        if (expiry.HasValue)  
            options.SetAbsoluteExpiration(expiry.Value);  
  
        _cache.Set(key, value, options);  
    }  
}
```

9. Testing

9.1 Backend - xUnit + Moq

```
// TriviUpTest/QuizServiceTests.cs  
  
public class QuizServiceTests
```

```
{

    private readonly Mock<IQuizRepository> _repositoryMock;

    private readonly QuizService _service;

    public QuizServiceTests()
    {
        _repositoryMock = new Mock<IQuizRepository>();

        _service = new QuizService(_repositoryMock.Object);
    }

    [Fact]

    public async Task CreateAsync_WithValidData_ReturnsQuiz()
    {
        // Arrange

        var request = new CreateQuizRequest
        {
            Nombre = "Test Quiz",

            EsPublico = true,

            Preguntas = new List<CreateQuestionRequest>()
        };

        _repositoryMock

            .Setup(r => r.SaveAsync(It.IsAny<Quiz>()))

            .ReturnsAsync((Quiz q) => { q.Id = 1; return q; });
    }
}
```

```
// Act

var result = await _service.CreateAsync(request, userId: 1);

// Assert

Assert.True(result.IsSuccess);

Assert.Equal("Test Quiz", result.Value.Nombre);

}

[Fact]

public async Task CreateAsync_WithEmptyName_ReturnsValidationError()

{

    // Arrange

    var request = new CreateQuizRequest { Nombre = "" };

    // Act

    var result = await _service.CreateAsync(request, userId: 1);

    // Assert

    Assert.True(result.IsFailure);

    Assert.IsType<QuizValidationError>(result.Error);

}

}
```

9.2 E2E - Playwright

```
// e2e/auth.spec.ts

test.describe('Authentication', () => {
```

```
test('should register and login successfully', async ({ page }) => {  
  
    await page.goto('/auth');  
  
    // Register  
  
    await page.click('text=Registrarse');  
  
    await page.fill('input[placeholder*="usuario"]', 'testuser');  
  
    await page.fill('input[placeholder*="email"]', 'test@test.com');  
  
    await page.fill('input[placeholder*="••"]', 'password123');  
  
    await page.click('button:has-text("Crear Cuenta")');  
  
    // Verify logged in  
  
    await expect(page.locator('text=testuser')).toBeVisible();  
  
});
```

9.3 Ejecutar Tests

```
# Backend  
  
cd TriviUpBackend/TriviUpTest  
  
dotnet test --configuration Release --collect:"XPlat Code Coverage"  
  
# Frontend  
  
cd TriviUpFrontend  
  
npm test -- --run  
  
# E2E  
  
cd TriviUpBackend/TriviUpE2ETests  
  
dotnet test
```

10. Despliegue

10.1 Railway

Servicios necesarios:

1. **PostgreSQL** - Base de datos
2. **Backend** - API .NET 9
3. **Frontend** - Angular + nginx

Variables de entorno RAILWAY (Backend):

Variable	Valor
ASPNETCORE_ENVIRONMENT	Production
Database__Provider	PostgreSQL
DATABASE_URL	(de Railway PostgreSQL)
Jwt__Key	(generar 32+ chars)
Jwt__Issuer	TriviUpBackend
Jwt__Audience	TriviUpApp
ALLOWED_ORIGINS	https://triviup.up.railway.app
FRONTEND_URL	https://triviup.up.railway.app
GOOGLE_CLIENT_ID	(de Google Cloud)
GOOGLE_CLIENT_SECRET	(de Google Cloud)

10.2 nginx Configuration

```
# TriviUpFrontend/nginx.conf

worker_processes auto;

error_log /var/log/nginx/error.log warn;
```

```
pid /var/run/nginx.pid;

events {

    worker_connections 1024;

}

http {

    include /etc/nginx/mime.types;

    default_type application/octet-stream;

    # Logging

    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    # Gzip

    gzip on;

    gzip_types text/plain text/css application/json application/javascript;

    # SPA fallback

    server {

        listen 80;

        server_name localhost;

        root /usr/share/nginx/html;

        index index.html;
```

```
# Security headers

add_header X-Frame-Options "SAMEORIGIN";

add_header X-Content-Type-Options "nosniff";

# SPA routing

location / {

    try_files $uri $uri/ /index.html;

}

# Proxy API

location /api/ {

    proxy_pass http://triviup-backend.railway.internal:5164;

    proxy_http_version 1.1;

    proxy_set_header Host $host;

    proxy_set_header X-Real-IP $remote_addr;

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

    proxy_set_header X-Forwarded-Proto $scheme;

}

# Proxy SignalR

location /hubs/ {

    proxy_pass http://triviup-backend.railway.internal:5164;

    proxy_http_version 1.1;

    proxy_set_header Upgrade $http_upgrade;
```

```
    proxy_set_header Connection "upgrade";

    proxy_set_header Host $host;

    proxy_set_header X-Real-IP $remote_addr;

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

# Proxy Auth

location /auth/ {

    proxy_pass http://triviup-backend.railway.internal:5164;

}

# Proxy Storage

location /storage/ {

    proxy_pass http://triviup-backend.railway.internal:5164;

}

# Proxy Users (necesario para admin)

location /users {

    proxy_pass http://triviup-backend.railway.internal:5164;

}

location /users/ {

    proxy_pass http://triviup-backend.railway.internal:5164;

}

}
```

```
}
```

10.3 Logs

```
# Ver logs del backend en Railway  
  
railway logs -s triviup-backend  
  
# Ver logs del frontend  
  
railway logs -s triviup-frontend
```

Anexo A: Glosario

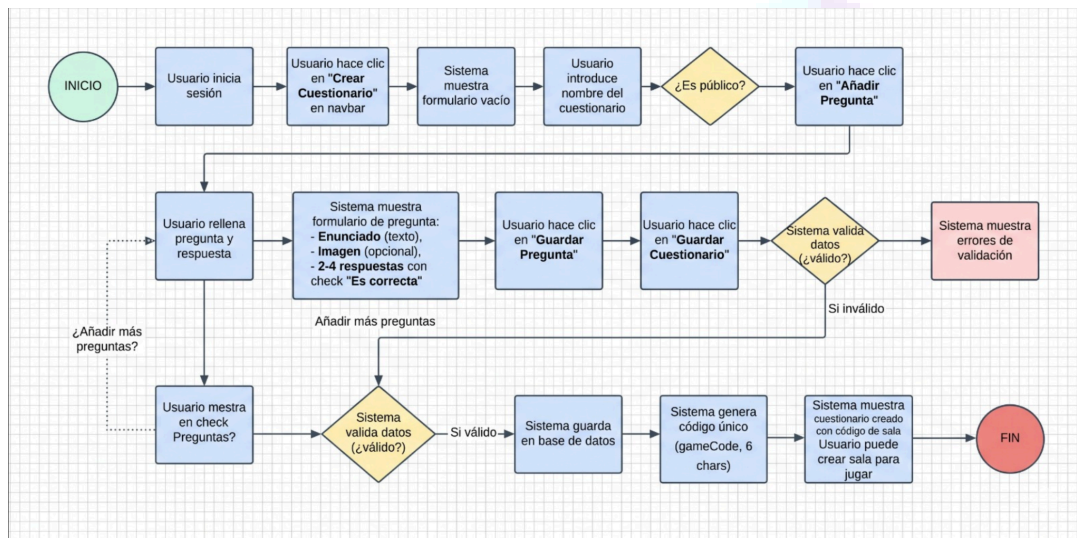
Término	Definición
SPA	Single Page Application - Aplicación web que carga una sola página
SignalR	Biblioteca .NET para comunicación en tiempo real via WebSockets
JWT	JSON Web Token - Estándar para autenticación stateless
EF Core	Entity Framework Core - ORM de Microsoft para .NET
Kestrel	Servidor HTTP cross-platform de ASP.NET Core
Clean Architecture	Patrón de arquitectura con capas separadas
Result Pattern	Patrón de manejo de errores con tipos explícitos

Anexo B: Referencia de API Endpoints

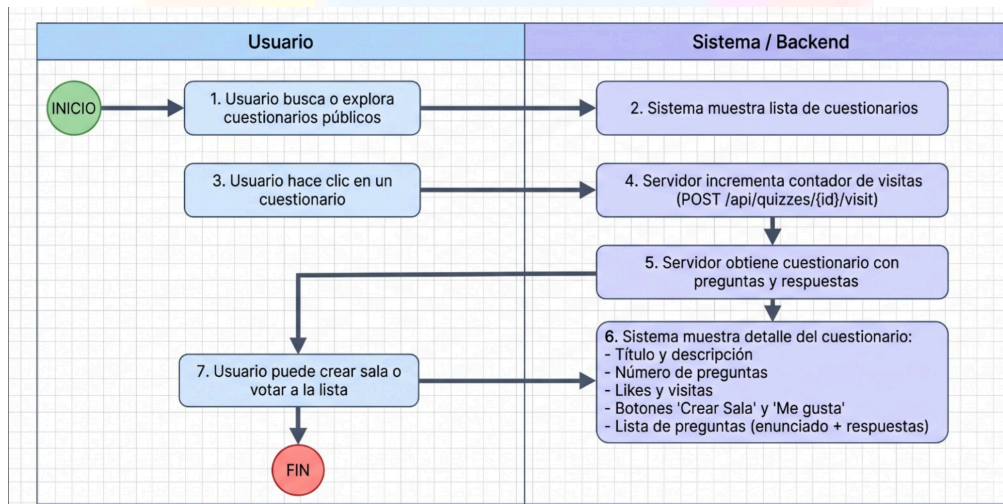
Ver Documentacion_Triviup_Cortes_Gomez.pdf sección API Reference para tabla completa.

Anexo C: Otros flujos

Flujo de creación de cuestionario



Flujo de vista de cuestionario



Flujo de dar like

